第23章 最小生成樹：詳解

23-E01 尊重割與並列的量詞

推導

跨S恰為cd3、be4、bd5、ce6；ac、bc在S內，de在補集內，因此A無跨邊。取含A的MST T；若cd不在T，T+cd的cycle必另有跨邊f，且f∉A。刪f後仍為樹，w(f)≥3，所以新樹不增成本且仍含A，證明cd safe。

三角形頂點x,y,z，各權1，MST為{xy,yz}、{xy,xz}、{xz,yz}，成本皆2。每條邊都是某割light並出現在其中兩棵MST，故第一句真；例如xy不在第三棵，第二句假；三條合起來是cycle，第三句假。

若e為cycle唯一最重而某MST含e，刪e形成割。沿原cycle的其餘路徑必有跨邊f，且w(f)<w(e)。交換後仍連通無環卻更便宜，與MST矛盾。

結論與檢核

本題safe的是cd:3。Light的「某棵」不能換成「每棵」；唯有割上唯一light才具有必選保證，cycle上唯一最重才具有全排除保證。

23-E02 Kruskal 的全部接受與拒絕

推導

原排序為bc1(ID2)、ac2(ID1)、de2(ID6)、cd3(ID4)、ab4(ID0)、be4(ID7)、bd5(ID3)、ce6(ID5)。依序分量與成本為：

邊	掃描前兩端	決定／處理後分量	成本
bc1	{b},{c}	接受；{a},{bc},{d},{e}	1
ac2	{a},{bc}	接受；{abc},{d},{e}	3
de2	{d},{e}	接受；{abc},{de}	5
cd3	{abc},{de}	接受；{abcde}	8
ab4	{abcde},{abcde}	拒絕；不變	8
be4	同上	拒絕；不變	8
bd5	同上	拒絕；不變	8
ce6	同上	拒絕；不變	8

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第23章 最小生成樹：詳解 （續）

已知連通時，第4次掃描接受第n−1邊即可停。改be1後排序為bc1、be1、ac2、de2、cd3、ab4、bd5、ce6；接受前四邊，分量依序{bc}、{bce}、{abce}、{abcde}，成本1、2、4、6，其餘皆拒。兩條權1或權2若交換先後，所選仍是respect割的light，最優性成立，不依賴ID。

結論與檢核

原成本8，降低後成本6。拒絕邊的兩端在同一**目前A分量**；不能改成一開始就檢查原圖連通分量，否則原連通圖所有邊都會被拒。

23-E03 不同圖模型的成本

推導

一般Kruskal為O(m log(m+1)+(n+m)(1+α(n)))；1保留讀入、MAKE-SET及每次操作的常數成本，亦適用本課α(2)=0的定義。固定n=1000時1+α(1000)=Θ(α(1000))，故以下數值案例可簡寫。simple案例代入O(2000 log2001+3000α(1000))；multigraph代入O(10^7log(10^7+1)+(10^7+1000)α(1000))。後者平行邊使m可遠大於n^2，無法從simple bound推出log m=O(log n)。這是漸近假設差異，固定兩個數字本身不能證明函數級關係。

Binary Prim分別O(3000log1000)、O((10^7+1000)log1000)；Fibonacci分別O(2000+1000log1000)、O(10^7+1000log1000)；線性掃key配原邊list為O(10^6+2000)、O(10^6+10^7)。若先對平行邊建matrix，只留每對最小權，讀完仍需Ω(m)，之後才可跑Θ(n^2) matrix Prim。

Counting sort成本O(m+10)，需另加O((n+m)(1+α(n)))的DSU及讀入成本。「不成cycle」只證A仍可延伸為某棵樹，未證可延伸為最優樹；還須證接受邊對當前某分量cut為light，再用交換定理。

結論與檢核

三種Prim的priority queue契約不同，不可用binary實作卻報Fibonacci成本。負權不影響比較算法；本題counting假設則明列0..9。

第23章 最小生成樹：詳解 （續）

23-E04 Prim 的競爭候選

推導

初始化S=∅，key[a]=0，其餘∞。逐輪掃完後：

extract／S	未取key(parent)	A／成本
a0／{a}	b4(a),c2(a),d∞,e∞	∅／0
c2／{a,c}	b1(c),d3(c),e6(c)	ac／2
b1／{a,b,c}	d3(c),e4(b)	ac,cb／3
d3／{a,b,c,d}	e2(d)	ac,cb,cd／6
e2／全部	無	ac,cb,cd,de／8

b由∞→4(parent a)→1(parent c)；e由∞→6(parent c)→4(parent b)→2(parent d)。extract c2後接b1已是下降；d3後接e2也下降。新加入頂點改變候選割，並無不變量要求key單調。

改成key[u]+w(u,v)是在累加從根來的路長。三角形取a後b4,c5，取b時Prim以bc2更新c為2、parent b；路徑算法候選4+2=6不優於5，保留parent a。兩棵樹分別成本6與9；從a到c的距離分別為6與5。

結論與檢核

本五點圖Prim成本8，parent[c]=a、parent[b]=c、parent[d]=c、parent[e]=d。Key代表一條跨邊，不能把其值解讀成a到該點的距離。

23-E05 證書、唯一性與權重降低

推導

非樹邊	T路	M_T	w−M_T
ab4	a-c-b，2,1	2	2
bd5	b-c-d，1,3	3	2
ce6	c-d-e，3,2	3	3
be4	b-c-d-e，1,3,2	3	1

全嚴格，故T唯一。be降到3時w′=M=3，舊T成本8仍最優，換cd為be得{bc,ac,de,be}也成本8，所以不唯一。降到1時同樣換邊，成本6；新樹非樹邊cd的路c-b-e-d最大2<3，ab路a-c-b最大2<4，bd路b-e-d最大2<5，ce路c-b-e最大1<6，皆嚴格，故新MST唯一。

第23章 最小生成樹：詳解 （續）

一般令原T成本W。若新樹R不含e，其權未改，w(R)≥W。若R含e，刪e分出割；T的e兩端路上必有跨邊g，且w(g)≤M。R−e+g是原圖生成樹，因此W≤w(R)−w′+w(g)≤w(R)−w′+M，得w(R)≥W+w′−M。綜合下界W+min(0,w′−M)，由保留T或換最大邊的構造達到。

結論與檢核

降至3：最優8、不唯一；降至1：最優6、唯一。降低的是一條非自環的原非樹邊，不能不改前提就套在樹邊增權或多邊同時變動。

23-E06 負權、平行邊與孤立點

推導

排序e0−100、e1−2、e3 0、e2 1、e4 4。e0兩端同a，拒絕；e1合a,b，成本−2；e3把c合入，成本仍−2；e2兩端已連通，拒絕；e4亦拒絕。結果邊{e1,e3}，原連通分量{abc}與{d}分別得到樹與空樹。

Prim從a依次取a0、b−2、c0；之後剩d∞。連通版應此時回報不連通，不能交付生成樹。Forest版將d當新根，令key[d]=0且parent[d]=NIL，不加邊。∞只表示不存在跨邊；負自環雖便宜卻不連接不同頂點且形成cycle。

結論與檢核

MSF總成本−2，共n−原分量數=4−2=2條邊；全圖MST不存在。兩條ab平行邊有不同ID，輸出e1，不因相同端點把兩條混為一條。

23-E07 Cut 證書的反向推導

推導

本題的證書與唯一性只對非自環非樹邊量化；自環不屬任何生成樹，無須取空路最大值。若MST T有這種邊w(e)<M_T，沿T+e的cycle刪一條最大邊便嚴格改善，矛盾。反向，對每個f∈T，刪f分割；任一其他跨邊e的T路必經f，所以w(e)≥M_T≥w(f)。f因此是該割light。逐條建T的任何prefix都respect這個割，安全邊定理反覆維持可最優延伸，所以T最優。

若某非樹邊等號，換最大邊便給另一棵同成本樹；若全嚴格，每個T−f割的其他跨邊都嚴格重於f，f為唯一light，任何MST必含f。所有T邊皆必含，故唯一。

對T′={ab1,ac2}，非樹bc1的T路b-a-c最大2，違反1≥2。換出ac2、換入bc1，得成本2的{ab,bc}。此三角形輸入權1,1,2有重複，但非樹ac2的路最大1，嚴格2>1，所以該MST唯一。

第23章 最小生成樹：詳解 （續）

結論與檢核

候選首先必須是一棵生成樹，否則M_T未必有唯一路徑。全異權是唯一性的充分條件，並非必要條件；正確的充要條件在非樹邊與樹路的比較。

23-E08 同成本交換與第2小邊

跨割邊為ad:2、bc:2、ac:4、bd:5；ab與cd各在一側內，所以割respect A。每棵生成樹需三條邊，最小三個權重為1、1、2，總權至少4。T連通且無環，成本4，達到下界。

加入bc:2形成a-b-c-d-a。可刪另一跨邊ad:2；它不在A內，因此A保留。新樹T′={ab,bc,cd}連通、無環且成本4−2+2=4，仍是MST。不能推出bc屬每棵MST：原本的T就是不含bc的MST。

對連通、無自環且無平行邊的圖，排序第2條邊e之前最多一條邊，故嚴格輕於e者至多一條。若最優樹未含e，加e後的唯一環長至少3；另兩條中至少一條f不比e輕。刪f仍為生成樹，權不增，故仍最優且包含e。同權時按任一固定次序排名，不影響「嚴格更輕至多一條」的論證。

平行邊反例：兩頂點a、b有權1與2的兩條邊。唯一MST取權1，排序第2條不屬任何MST。這不是割定理失效，而是原證明用到的「環長至少3」不再成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第23章 最小生成樹：詳解 （續）

原創驗算程式附錄: kruskal_mst.py

```
"""Kruskal MST with a compact disjoint-set implementation."""

def mst(vertices, edges):
    parent = {v: v for v in vertices}
    rank = {v: 0 for v in vertices}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        if rank[ra] < rank[rb]:
            ra, rb = rb, ra
        parent[rb] = ra
        if rank[ra] == rank[rb]:
            rank[ra] += 1
        return True

    chosen = []
    for weight, u, v in sorted(edges):
        if union(u, v):
            chosen.append((weight, u, v))
    return chosen


edges = [(2, "a", "c"), (5, "a", "d"), (1, "b", "c"), (4, "b", "d")]
chosen = mst({"a", "b", "c", "d"}, edges)
assert len(chosen) == 3 and sum(edge[0] for edge in chosen) == 7
print(chosen)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第23章 最小生成樹：詳解 （續）

原創驗算程式附錄: mst_checks.py

```
"""Compare Kruskal with exhaustive spanning trees on small graphs."""

from itertools import combinations
import random


class DisjointSet:
    def __init__(self, vertices):
        self.parent = {vertex: vertex for vertex in vertices}

    def find(self, item):
        while self.parent[item] != item:
            self.parent[item] = self.parent[self.parent[item]]
            item = self.parent[item]
        return item

    def union(self, left, right):
        left, right = self.find(left), self.find(right)
        if left == right:
            return False
        self.parent[left] = right
        return True


def kruskal(vertices, edges):
    result = []
    sets = DisjointSet(vertices)
    for weight, u, v in sorted(edges):
        if sets.union(u, v):
            result.append((weight, u, v))
    return result


def is_spanning_tree(vertices, edges):
    if len(edges) != len(vertices) - 1:
        return False
    sets = DisjointSet(vertices)
    return all(sets.union(u, v) for _, u, v in edges)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第23章 最小生成樹：詳解 （續）

```
def brute_force_mst(vertices, edges):
    candidates = [
        selected
        for selected in combinations(edges, len(vertices) - 1)
        if is_spanning_tree(vertices, selected)
    ]
    return min((sum(edge[0] for edge in tree) for tree in candidates), default=None)


def run():
    triangle = [(1, "a", "b"), (3, "a", "c"), (2, "b", "c")]
    result = kruskal(["a", "b", "c"], triangle)
    assert sum(edge[0] for edge in result) == 3
    assert brute_force_mst(["a", "b", "c"], triangle) == 3

    rng = random.Random(23)
    trials = 0
    for size in range(3, 8):
        vertices = [f"v{i}" for i in range(size)]
        for _ in range(15):
            edges = []
            for i in range(size):
                for j in range(i + 1, size):
                    if rng.random() < 0.45:
                        edges.append((rng.randint(-3, 8), vertices[i], vertices[j]))
            if brute_force_mst(vertices, edges) is None:
                continue
            tree = kruskal(vertices, edges)
            assert is_spanning_tree(vertices, tree)
            assert sum(edge[0] for edge in tree) == brute_force_mst(vertices, edges)
            trials += 1
    assert trials > 20
    print(f"PASS triangle_MST=3 brute_force_cases={trials} negative_weights=allowed")


if __name__ == "__main__":
    run()
```

第23章 最小生成樹：詳解 （續）

原創驗算程式附錄: mst_depth_checks.py

```
"""MST/MSF behavior and certificates versus exhaustive spanning-forest subsets."""
from itertools import combinations,product
from math import inf
import random

SAMPLE=[(0,1,4),(0,2,2),(1,2,1),(1,3,5),(2,3,3),(2,4,6),(3,4,2),(1,4,4)]


class DSU:
    def __init__(self,n):self.parent=list(range(n));self.rank=[0]*n
    def find(self,x):
        if x!=self.parent[x]:self.parent[x]=self.find(self.parent[x])
        return self.parent[x]
    def union(self,a,b):
        a,b=self.find(a),self.find(b)
        if a==b:return False
        if self.rank[a]<self.rank[b]:a,b=b,a
        self.parent[b]=a
        if self.rank[a]==self.rank[b]:self.rank[a]+=1
        return True
    def groups(self):return frozenset(frozenset(i for i in range(len(self.parent)) if
self.find(i)==root) for root in set(self.find(i) for i in range(len(self.parent))))


def kruskal(n,edges):
    d=DSU(n);selected=[];trace=[]
    for identity in sorted(range(len(edges)),key=lambda i:(edges[i][2],i)):
        u,v,w=edges[identity];accepted=d.union(u,v)
        if accepted:selected.append(identity)
        trace.append((identity,accepted,d.groups()))
    return selected,trace


def prim(n,edges):
    # Linear minimum selection implements the documented dense variant, not a heap bound.
    adjacency=[[] for _ in range(n)]
    for identity,(u,v,w) in enumerate(edges):
        adjacency[u].append((v,w,identity));adjacency[v].append((u,w,identity))
    key=[inf]*n;parent=[None]*n;remaining=set(range(n));selected=[];trace=[]
```

第23章 最小生成樹：詳解 （續）

```
    if n:key[0]=0
    while remaining:
        u=min(remaining,key=lambda v:(key[v],v))
        if key[u]==inf:key[u]=0  # Explicitly start another connected component.
        extracted=key[u];remaining.remove(u)
        if parent[u] is not None:selected.append(parent[u])
        for v,w,identity in adjacency[u]:
            if v in remaining and w<key[v]:key[v]=w;parent[v]=identity
        trace.append((u,extracted,key[:],parent[:]))
    return selected,trace


def forest(n,edges,selected):
    d=DSU(n)
    for identity in selected:
        u,v,w=edges[identity]
        if not d.union(u,v):return None
    return d.groups()


def path_max(n,edges,selected,source,target):
    if source==target:return -inf
    adjacency=[[] for _ in range(n)]
    for identity in selected:
        u,v,w=edges[identity];adjacency[u].append((v,w));adjacency[v].append((u,w))
    pending=[(source,None,-inf)]
    while pending:
        u,parent,maximum=pending.pop()
        if u==target:return maximum
        for v,w in adjacency[u]:
            if v!=parent:pending.append((v,u,max(maximum,w)))
    return inf


def certificate(n,edges,selected,strict=False):
    allsets=DSU(n)
    for u,v,w in edges:allsets.union(u,v)
    if forest(n,edges,selected)!=allsets.groups():return False
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第23章 最小生成樹：詳解 （續）

```
    chosen=set(selected)
    for identity,(u,v,w) in enumerate(edges):
        if identity in chosen or u==v:continue
        maximum=path_max(n,edges,selected,u,v)
        if w<maximum or (strict and w==maximum):return False
    return True


def oracle(n,edges):
    target=DSU(n)
    for u,v,w in edges:target.union(u,v)
    partition=target.groups();needed=n-len(partition);best=inf;optimal=[];candidates=[]
    for chosen in combinations(range(len(edges)),needed):
        if forest(n,edges,chosen)!=partition:continue
        value=sum(edges[i][2] for i in chosen);candidates.append((chosen,value))
        if value<best:best=value;optimal=[frozenset(chosen)]
        elif value==best:optimal.append(frozenset(chosen))
    return best,optimal,candidates


def verify(n,edges):
    best,optimal,candidates=oracle(n,edges)
    for chosen,_ in [kruskal(n,edges),prim(n,edges)]:
        assert frozenset(chosen) in optimal and sum(edges[i][2] for i in chosen)==best
        assert certificate(n,edges,chosen)
        assert certificate(n,edges,chosen,True)==(len(optimal)==1)
    for chosen,value in candidates:assert certificate(n,edges,chosen)==(value==best)
    # A unique light edge across a cut belongs to every optimum, not merely some.
    for mask in range(1,(1<<n)-1):
        crossing=[i for i,(u,v,w) in enumerate(edges) if ((mask>>u)&1)!=((mask>>v)&1)]
        if not crossing:continue
        minimum=min(edges[i][2] for i in crossing)
        light=[i for i in crossing if edges[i][2]==minimum]
        if len(light)==1:assert all(light[0] in chosen for chosen in optimal)
    return len(candidates)


def main():
```

第23章 最小生成樹：詳解 （續）

```
    graphs=0;candidates=0
    for n in range(5):
        pairs=list(combinations(range(n),2))
        for weights in product([None,-1,0,2],repeat=len(pairs)):
            edges=[(u,v,w) for (u,v),w in zip(pairs,weights) if w is not None]
            candidates+=verify(n,edges);graphs+=1
    rng=random.Random(23)
    for _ in range(150):
        n=rng.randrange(2,7);pairs=list(combinations(range(n),2));rng.shuffle(pairs)
        edges=[(u,v,rng.randrange(-4,6)) for u,v in pairs[:min(9,len(pairs))]]
        candidates+=verify(n,edges);graphs+=1
    assert oracle(5,SAMPLE)[0]==8
    chosen,trace=kruskal(5,SAMPLE);assert chosen==[2,1,6,4]
    pt,ptrace=prim(5,SAMPLE);assert pt==[1,2,4,6]
    assert [(u,key) for u,key,*_ in ptrace]==[(0,0),(2,2),(1,1),(3,3),(4,2)]
    triangle=[(0,1,1),(1,2,1),(0,2,1)];verify(3,triangle)
    assert len(oracle(3,triangle)[1])==3
    multigraph=[(0,0,-100),(0,1,2),(0,1,-1),(1,2,0),(0,2,3),(3,4,-2)]
    verify(6,multigraph)
    assert sum(multigraph[i][2] for i in kruskal(6,multigraph)[0])==-3
    # Sensitivity: decrease outside edge; swap its cycle's maximum iff now cheaper.
    for identity,(u,v,w) in enumerate(SAMPLE):
        if identity in chosen:continue
        for reduced in range(-2,w+1):
            changed=SAMPLE[:];changed[identity]=(u,v,reduced)
            maximum=path_max(5,SAMPLE,chosen,u,v)
            best,opts,_=oracle(5,changed)
            assert best==8+min(0,reduced-maximum)
    print(f'PASS weighted_graphs={graphs} spanning_forest_candidates={candidates} '
          'Kruskal_and_dense_Prim=exhaustive_oracle certificate_iff_optimal=True '
          'unique_iff_strict_path_max=True cuts_unique_light_mandatory=True '
          'negative_parallel_selfloop_disconnected=True sample_weight=8 Prim_keys=0,2,1,3,2')


if __name__=='__main__':main()
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study